

Forkable Sandboxes

A Runtime Layer for AI Software Factories

N forks $\rightarrow N$ executions, but $N_{\text{eff}} < N$ independent observations.

Yossi Eliaz

Principal Engineer, Incredibuild · Associate Professor, Holon Institute of Technology (HIT)

Disclosure: I work for a company that builds sandboxes.



Hedged requests reduce tail latency

If a server is slow, ask a second server too, and use whichever answers first.

1,800 ms $\rightarrow$ 74 ms

Latency of the slowest 0.1% of requests. The backup requests add 2% to the load.

“the source of latency is often not inherent in the particular request”: the server is slow, not the request.

Hedged requests reduce tail latency

If a server is slow, ask a second server too, and use whichever answers first.

1,800 ms $\rightarrow$ 74 ms

Latency of the slowest 0.1% of requests. The backup requests add 2% to the load.

“the source of latency is often not inherent in the particular request”: the server is slow, not the request.

Agents hedge for correctness. A flaw in the task repeats in every copy.

Dean & Barroso, The Tail at Scale, CACM 2013, backup after 10 ms, 99.9th-percentile latency reading 1,000 BigTable keys. It works only when the slowdown does not hit several servers at once. Shared mistakes across different AI models: [Kim et al., ICML 2025](#).

Repeated sampling needs an automatic check

16% → 56%

SWE-bench Lite, one sample vs. 250
(Brown et al. 2024)

- ✓ with an automatic check, more samples solve more problems
- ✗ without one, majority voting and reward models stop improving
- ✗ a check that accepts wrong answers caps the gain, and when a wrong answer costs more than none, Stroebl et al. (2024) find the best number of tries is often under ten

Hedging for correctness improves accuracy only as far as the check is reliable.

Brown et al., Large Language Monkeys: Scaling Inference Compute with Repeated Sampling, arXiv:2407.21787;
Stroebl, Kapoor & Narayanan, The Limits of Inference Scaling Through Resampling (v1: Inference Scaling fLaws), arXiv:2411.17501.

Latency and scale of agent runtimes

Operation	Number	Source
checkpoint an agent sandbox	10.8 ms	DeltaBox, 2026
resume a paused microVM	49 ms, best case	Kimi K3, 2026
boot a microVM to application code	<125 ms	Firecracker, NSDI'20
fork an agent sandbox	134–143 ms	Shepherd, 2026
fork a Xen VM across hosts	600–800 ms	SnowFlock, EuroSys'09
restore, run, capture through a public API	6.87 s p50	Eliaz, arXiv:2607.09689, Table 2
one work order in our experiments	43 min, \$10.33	our run, 27 Sep 2026
a sandbox's life spent waiting on the model	up to 98%	Kimi K3, 2026
sandboxes across one model's training and evaluation	51.2 million	Kimi K3, 2026

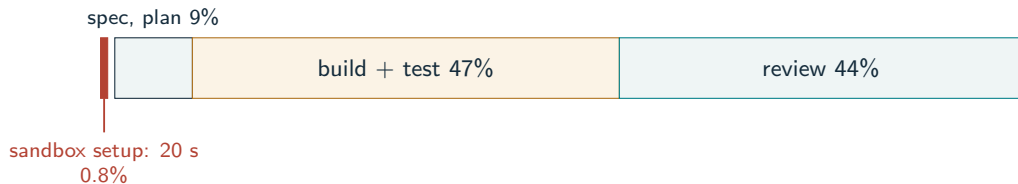
how correlated two forks' verdicts are

? no surveyed system reports it

No surveyed system reports the correlation between two forks' verdicts.

DeltaBox, Kimi K3 report §5.3.2, Agache et al. NSDI'20, Shepherd, Lagar-Cavilla et al. EuroSys'09, arXiv:2607.09689, our experiments, Sep 2026 (time, cost). Survey, slide 22. After J. Dean, "Numbers everyone should know" (LADIS 2009).

Where 43 minutes went



A hundredfold faster sandbox setup would shorten this run by 0.8%.

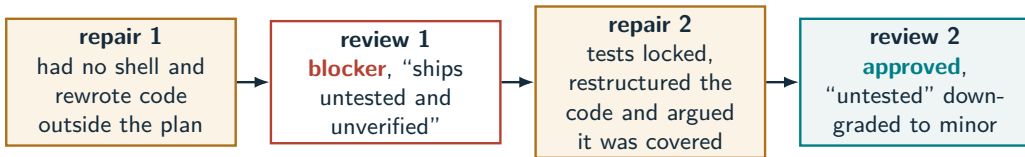
ANALOGY A self-evolving loop, an agent system that proposes, tests and merges changes to its own code, is self-driving software that takes an issue and produces a reviewed patch.

Both the minutes and the risk lie in the agents and checks.

Our experiments, Sep 2026, stage timings (one cycle 2,557 s, sandbox setup 20.3 s).

A review gate approved untested code

The work order was to fix the sandbox allowlist. A four-thread race test failed, and the loop sent a repair agent.

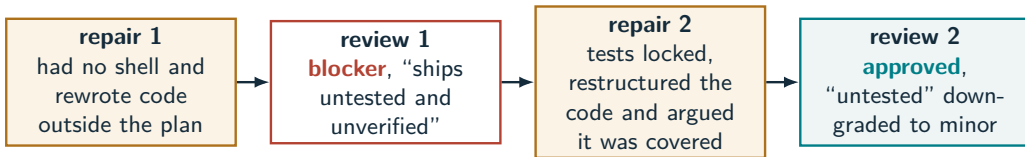


One model performed all four steps. All 1,972 tests passed after each repair. The new code had no test of its own.

Both repairs said they had not run the tests. The gate approved.

A review gate approved untested code

The work order was to fix the sandbox allowlist. A four-thread race test failed, and the loop sent a repair agent.



One model performed all four steps. All 1,972 tests passed after each repair. The new code had no test of its own.

Both repairs said they had not run the tests. The gate approved.

Suppose nine forks of repair 1 all pass. We ask how many independent observations that provides.

Our experiments, Sep 2026, agent records of both repairs and both reviews.

One question across six fields

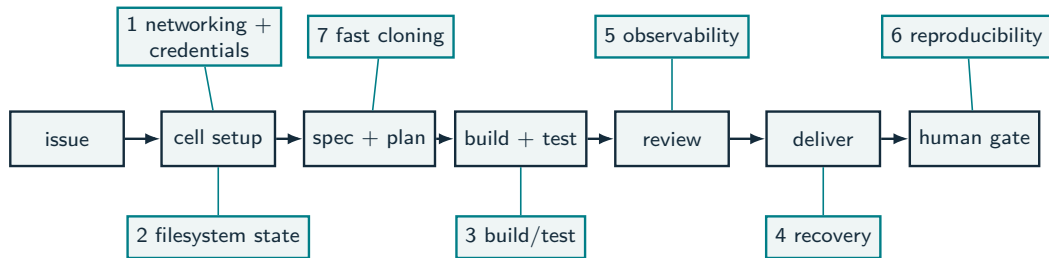
In each of six fields, the question is how many independent observations exist.



In each field, apparently independent evidence is not independent.

Physics and biology give vocabulary only, no evidence about forks. Genomics and Bitcoin are team results with co-authors. Physics: PRE 2020, PNAS 2020, JPCB 2021. Genomics: Hitz et al. 2023, iScience 2026. Bitcoin: arXiv:2206.02871. Self-driving: perception work at Mobileye, 2021–2024. Agent loops: our experiments, arXiv:2607.09689. RL: Kimi K3 report §5.3.2.

Seven runtime demands in one pipeline



MEASURED our run records **BUILT** built and tested, or synthetic **PROPOSED** design, not yet run
PUBLISHED other people's work **ANALOGY** borrowed vocabulary only

Scope. We ran two experiments with the loop in Docker Sandboxes through Airflow's sandbox toolset, with models served through Databricks. Two observations come from our own sandbox platform. Our harness answered the gates. Leases were off. No run forked.

Intelligence as choice and action

Intelligence derives from Latin *intellegere*, from *inter* (between) and *legere* (to choose, to read).

Intelligence means choosing between.

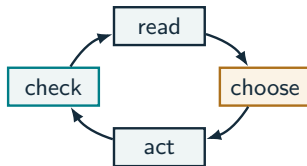
Since 29 Sep 2026, US federal agencies must call AI 'Super Intelligence' (SI). We say intelligence.

ANALOGY This ladder is our own framing.

self-driving cars sense, plan and act (Mobileye)

self-driving computers write and test their own software
(our experiments)

self-driving intelligence is an agent in a loop

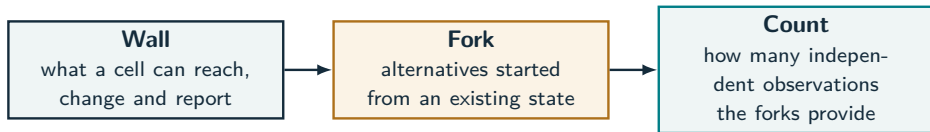


Act happens in a sandbox (**wall**). **Choose** needs alternatives (**fork**). **Check** decides what counts (**count**).

Intelligence acts on what it reads, and it acts inside the sandbox.

Oxford English Dictionary / Online Etymology Dictionary, 'intelligence'. Executive order 'Inaugurating the Era of Super Intelligence', 29 Sep 2026 (White House fact sheet; CNBC, 29 Sep 2026). Mobileye perception work, 2021–2024.

The loop needs a wall, a fork and a count



We claim

BUILT a receipt contract in which reused evidence stops the merge and each result carries its lineage, a record of its origin

MEASURED two experiments with the loop

PROPOSED H1 snapshot restore beats a warm cache by $>10s$

PROPOSED H2 siblings from one snapshot fail together more than strangers

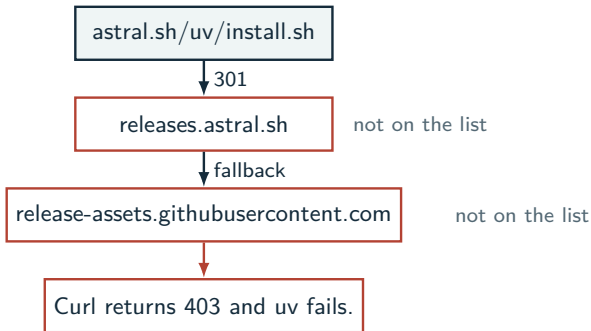
We do not claim

- ✗ a new estimator
- ✗ a controlled speedup
- ✗ a ranking of platforms
- ✗ a loop that forks today
- ✗ physics or biology as evidence about forks

 N forks $\rightarrow N$ executions, but $N_{\text{eff}} < N$ independent observations.

The runtime sees what forks share.

Allowlists fail when hosts redirect

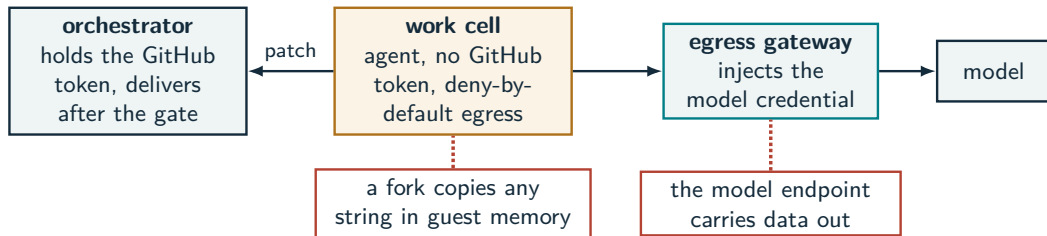


- The list lived in five places, two scripts, a checker, two docs.
- The fix added both hosts.
- The second host admits every release asset on GitHub.

We propose fetching files by digest through a caching proxy.

Observed on our sandbox platform, 27 Sep 2026. Work order from our first experiment, Sep 2026.

Forks copy cell memory, so keys stay outside



PROPOSED The gateway keys each request on host-assigned identity (per-clone tap or vsock CID) and ignores strings in guest memory. Restore bumps a generation and voids the parent's lease.

Credentials never belong in guest memory.

Our design notes on sandbox isolation and credential authority. [Docker Sandboxes security](#). Capsicum (USENIX Sec'10). Macaroons (NDSS'14). Gray & Cheriton, Leases (SOSP'89).

Filesystem state shared by a snapshot

State	Policy	Known mechanisms
files, build outputs	pin by content digest	OBuilder takes a copy-on-write snapshot per build step (ZFS, btrfs)
workspace disk	copy-on-write layers	overlay layers (day10), DeltaBox's layered copy-on-write filesystem and chained overlaybd snapshots (DeepSeek DSec)
agent history	keep outside the sandbox	
randomness	reseed, or couple deliberately	see slide 38
external services	mediate access	see slide 27

A snapshot captures only the state inside the sandbox boundary.

OBuilder; day10; DeltaBox; DeepSeek-V4 §5.2.5.

Write-protected tests did not limit edits

In our run

- ✓ tests/ write-protected, and the agent was told
- ✓ the test hook never fired, and all 15 edits were allowed
- ✗ planned steps are scope-checked, but repairs are not
- ✗ a repair edited code outside the plan
- ✗ an out-of-plan edit is rated “major”, not blocking
- ✗ the writer and both reviewers use one model

We propose scope-checking repairs and blocking out-of-plan edits.

A write-protected directory does not define the scope of permitted edits.

Our experiments, Sep 2026. Repair 1's record (an edit outside the plan). The loop's repair stage records repairs without a scope check. Barkai, . . . , Eliaz, . . . , OffRisk, *Bioinformatics Advances* 3:vbad138 (2023).

ANALOGY · BIOLOGY

The enzyme Cas9 can cut away from its guide, at an **off-target** site.

OffRisk, a Docker image we built with our co-authors, annotates CRISPR off-target sites and labels their risk (2023).

A scope check serves the same function for code.

Reinforcement learning calls it reward hacking

Source	What it found
METR (2025)	o3 reward-hacked in 30.4% of RE-Bench runs and 0.7% of HCAST runs, and METR's leading hypothesis is the visible scorer
ImpossibleBench	read-only tests prevent edits to the tests but not special-casing of the code for the tests
Kimi K3 §4.2.6	isolates agents from verifiers and pairs public diagnostic verifiers with hidden ones

On kernel-optimisation tasks (§4.2.4), the Kimi K3 hacking-detection system penalises “CUDA graph replay, input caching, and precision reduction”.

Caching inputs is the intended function of a build farm, but a speed grader treats it as a hack.

Running, evaluating and promoting code require three separate permissions.

METR (June 2025); ImpossibleBench; Kimi K3 technical report §4.2.4, §4.2.6.

Teardown destroyed the approved patch



The stray file was our own harness's review-diff archive, mirrored into the workspace. Our second work order took 26 min, cost \$3.43 and delivered nothing.

The gate refused correctly, and recovery lost the patch.

The artifact should leave the sandbox before it is destroyed. During a long human wait, the sandbox should be snapshotted, destroyed and restored.

Our second experiment, Sep 2026: run records (26 min wall clock, 853.8 s stage time, \$3.43); redacted copy on request; fix not yet merged. Related: Planarian statepoints (arXiv:2609.35366); Crab turn-level recovery (arXiv:2604.28138); DeepSeek-V4 DSec keeps the sandbox when its training task is preempted and replays cached results on resumption (§5.2.5).

Three of our check records were wrong

Sandbox evaluation CI job

[success](#) in 3s, with every real step skipped because the key was unset

Approval record

"mode": "human", actor admin, answered by our harness

Our sandbox CLI

status lines mixed into stdout, and an earlier attempt failed while parsing them

All three are our own receipts, records of what ran, and each was wrong.

Every check needs a receipt the sandbox cannot write.

Reinforcement learning applies the same rule. Kimi K3 grounds reward in the verifier's evaluation of the final environment state "rather than the agent's self-reported completion" (§4.2.6).

Our CI, 28 Sep 2026; our experiments, Sep 2026: the approval record and both repair records. Provenance: PASS (USENIX ATC'06), CamFlow (SoCC'17), in-toto (USENIX Security'19). [Kimi K3](#) §4.2.6.

What the run pins, and what it cannot

Pinned in the run record

- ✓ input commit (8ff967d) and `uv.lock`
- ✓ policy and inputs digests
- ✓ sandbox adapter digest
- ✓ every agent step's output, as JSON

Not pinned

- ✗ the hosted model, which has no sampling seed
- ✗ external hosts and their redirects
- ✗ the clock and scheduling
- ✗ the adapter source, a local shim outside git

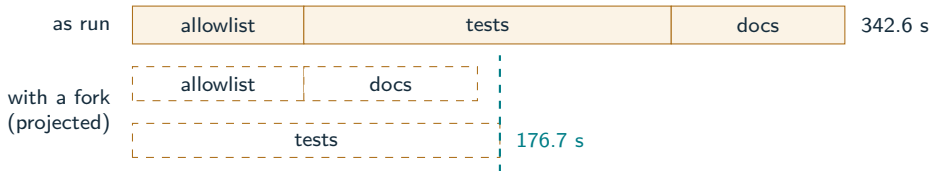
PROPOSED We propose to log every model response, external fetch and clock read, and to re-execute the rest against the log.

In genomics (slide 7), each ENCODE data file is recorded with the software versions, parameters and reference genome that produced it.

This run can be audited but cannot be rerun from its record.

Our experiments, Sep 2026: stage timings and work-graph execution record. [Hitz et al., The ENCODE Uniform Analysis Pipelines, bioRxiv 2023](#). Record and replay: `rr` (USENIX ATC'17); DeepSeek-V4 DSec logs every sandbox command and result for replay ([arXiv:2606.19348 §5.2.5](#)).

The parallel-safe steps had no fork available



reason: `serial_fallback_missing_fork` (the executor recorded that no fork was available)

The steps touch different files, so two separate git worktrees would suffice here. A fork matters when the reached state is expensive.

A fork would have saved about three of the forty-three minutes.

Our experiments, Sep 2026: work-graph execution record (node timings measured; the fork lane is computed, fork cost excluded). Remote fork at scale: MITOSIS (OSDI'23).

Six findings from two experiments

- | | |
|--|--|
| 1 Sandbox setup took little time | setup was under 1% of a 43-minute work order |
| 2 A locked test folder did not limit the repair | a repair rewrote code outside the plan |
| 3 Passing checks can test nothing | CI skipped every step; a review approved untested code |
| 4 One model performed all four steps | one model wrote, reviewed, fixed and approved |
| 5 Teardown can destroy the result | an approved patch was lost with its sandbox |
| 6 No fork was available | the loop requested a fork; none was offered |
-

Across seven surfaces, trusted components should run outside the sandbox.

Our experiments, Sep 2026: two work orders in Docker Sandboxes, models served through Databricks. Records: github.com/zozo123/ariflow-swfactory.

SnowFlock described agent sandboxing in 2009

(a) Sandboxing

```
state = trusted_code()
ID = VM_fork(1)
if ID.isChild():
    untrusted_code(state)
    VM_exit()
else:
    VM_wait(ID)
```

(b) Parallel Computation

```
ID = VM_fork(N)
if ID.isChild():
    parallel_work(data[ID])
    VM_exit()
else:
    VM_wait(ID)
```

In (b), `data[ID]` makes each child different. Forking an agent's repair nine times gives nine children and one slice.

Next, from Xen (2003) to training gyms (2026).

The mechanism is old, while the caller is a program that searches.

Lagar-Cavilla et al., SnowFlock: rapid virtual machine cloning for cloud computing, EuroSys'09, Fig. 1(a) and 1(b), built on Xen 3.0.3 (Barham et al., SOSP'03).

Fork systems since 2003, many built on Xen

Year	System
2003	Xen (Cambridge)
2005	Live migration (Cambridge)
2005	Potemkin
2009	SnowFlock
2020	Catalyzer
2023	Nephele, MITOSIS

Year	System
2026	DeltaBox
	Shepherd
	Crab
	Planarian
	Kimi K3 AgentENV (training)

Researchers have worked for twenty-three years to make fork cheap, and no surveyed system counts what sibling forks share.

SOSP'03; NSDI'05; SOSP'05; EuroSys'09; ASPLOS'20; EuroSys'23; OSDI'23; [DeltaBox](#); [Shepherd](#); [Crab](#); [Planarian](#); [Kimi K3](#) §5.3.2.

Training runs create sandboxes at large scale

51,219,741

sandboxes, across 1,505,678 images

Kimi K3, training + evaluation

- **pause** the sandbox while the model thinks, up to 98% of its life, using no memory or CPU
- **fork** from the exact state while the original keeps running, “for reward judging without side effects”
- **snapshot** at regular intervals, for error recovery
- **partial rollout**: optimise once a fraction λ of trajectories (attempts) completes, “to mitigate the long-tail latency”

DeepSeek-V4's DSec runs hundreds of thousands of concurrent sandboxes in one cluster.

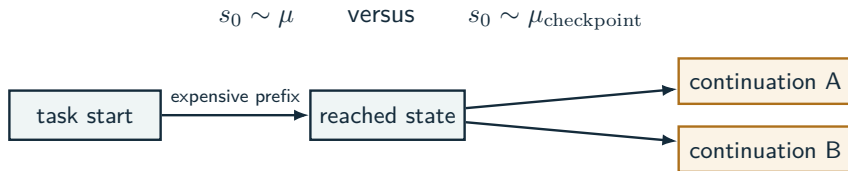
Partial rollouts follow Dean & Barroso's “good enough” approach of not waiting for the slowest attempts.

In training, a sandbox is the environment, a fork is a reset, and a verifier is the reward.

Kimi K3 §4.1.2, §5.3.2 (AgentENV); DeepSeek-AI, DeepSeek-V4, [arXiv:2606.19348](#) §5.2.5; Dean & Barroso, The tail at scale, CACM 2013.

Reset changes the learning problem

Where each attempt starts decides what the learner sees.



Go-Explore: first return, then explore (Ecoffet et al., Nature 2021)

- **PROPOSED** record each checkpoint's origin and the rule that selected it
- **PROPOSED** evaluate the final policy on the intended task distribution

The same applies to forking after the plan, because each child's success depends on it (slide 47).

Each restore point selects the learner's training data.

Ecoffet et al., First return, then explore, Nature (2021).

Fork copies memory, secrets and identity

The child inherits	Hazard	What is known
random-number state	siblings share random streams	VMGenID reseeds the kernel only, userspace has “no generic solution”
tokens in memory	N live copies	keep secrets outside, wipe on suspend (Brooker et al. 2021)
guest clock	resumes at snapshot time	resync on restore
IP and MAC	identical identities	namespace + NAT per clone, Potemkin spent 142.8 of 521 ms on IP setup
open TCP connections	cannot be cloned	CRIU hands one to one restore, gVisor returns ECONNRESET

Eight forks of a VM holding a token are eight live tokens.

Firecracker snapshotting docs; [Brooker et al.](#), [arXiv:2102.12892](#); Vrable et al. SOSP’05; CRIU and gVisor checkpoint/restore docs.

What each kind of fork copies

$M_{\text{total}} \approx M_{\text{shared}} + \sum_i M_{\text{private},i}$, as children share memory until they write.

Kimi K3, real workloads, copy-on-write memory plus page-cache optimisations, up to $6.5\times$ overcommit.

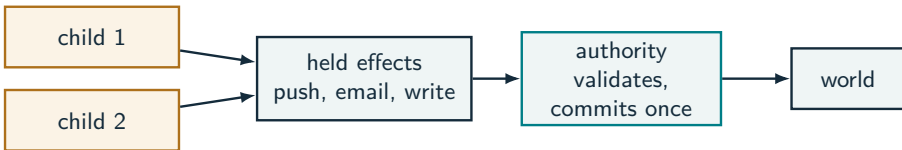
Mechanism	What survives	Where the cost lands
worktree, overlay layer	files only	bytes each child writes
container checkpoint (CRIU)	processes and memory, a TCP connection needs its original IP, so one copy at most	dump size, what the kernel can checkpoint
microVM snapshot	guest memory and vCPU/device state, disks handled separately	pages each child touches after restore
record and replay (rr)	a log of execution	replay time

A restore that returns OK is not a faithful copy. Before counting a child, compare restored and direct runs on declared outputs.

Fork is cheap, divergence is costly.

Firecracker snapshot-support docs; CRIU TCP_REPAIR docs; REAP (ASPLOS'21); rr (ATC'17); Kimi K3 §5.3.2.

Hold effects until an authority releases them



- Speculator, external synchrony and Remus run ahead and hold output until it is safe
- Zheng et al. (2026) state in Lean that an execution edit cannot undo a tool request already sent
- DSec replays cached results on resume, so a command unsafe to repeat never runs twice
- **BUILT** here: children hold no publishing credential, and the orchestrator delivers after the gate
- The one exception is the model API, an unheld channel that is metered and logged

Output commit is enforced by the system structure.

Speculator SOSP'05; external synchrony OSDI'06; Remus NSDI'08; Zheng et al., [arXiv:2608.22928](#); DeepSeek-V4 §5.2.5.

When fork pays, and where it loses

$$(N - 1) P > H + N (R + D)$$

Fork saves $N - 1$ preparations P and spends one capture H , plus a restore R and a divergence D per child Work W (model calls included) and evaluation V cancel, since each child still rebuilds its patch.

Where fork loses

- ✗ only one child, $N = 1$
- ✗ the state is only files and the build cache is good
- ✗ a minimal image boots faster (Jitsu, LightVM)

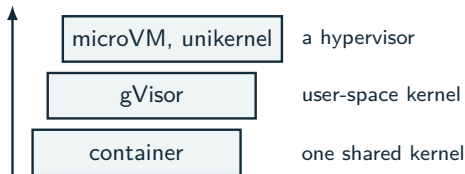
Fork loses when the build cache is good, and it pays only for state that cannot be rebuilt.

Eliaz, [arXiv:2607.09689](https://arxiv.org/abs/2607.09689), Table 2; Madhavapeddy et al. NSDI'15 (Jitsu); Manco et al. SOSP'17 (LightVM, NEC Labs).

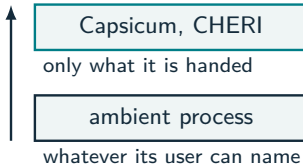
Isolation has two axes, kernel and authority

Kernel surface

less shared



Authority



PROPOSED Share warm state on purpose, a build cache within one trust domain (risk: timing and content leaks) · dependency image by digest (risk: a poisoned base) · scratch never (risk: cross-branch contamination)

Pick the kernel surface for the workload. Hand over authority explicitly.

Kimi K3 §5.3.2; Firecracker production host setup guide; Watson et al., Capsicum (USENIX Sec'10) and CHERI (IEEE S&P'15).

The interface the runtime owes the loop

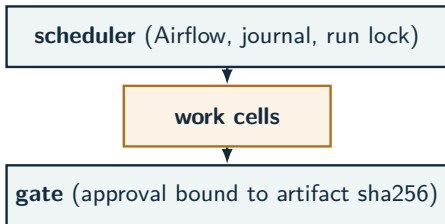
Call	Status
<code>checkpoint(cell) → snap</code>	MEASURED sandbox-platform named snapshots, memory unverified
<code>fork(snap, lease, seed=copy reseed, egress)</code>	PROPOSED
<code>evaluate(artifact, verifier) → receipt</code>	PROPOSED runs outside the child
<code>select(candidates, fresh test) → one</code>	PROPOSED selection without pooling
<code>reduce(receipts) → ($\hat{\theta}, N_{\text{eff}}$ bound) abstain</code>	BUILT merge + evidence check, PROPOSED N_{eff} bound, abstain
<code>promote(epoch, receipt)</code>	BUILT sha256-bound gate, PROPOSED epoch fence

The epoch fence stops a stale run from writing, like a Chubby sequencer or a fencing token.

Select chooses and reduce pools, so they are different calls.

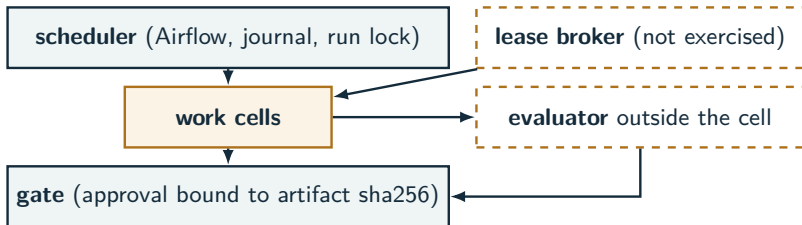
Fork contract (design document), sandbox-platform named snapshots: [arXiv:2607.09689](https://arxiv.org/abs/2607.09689) §4; Burrows OSDI'06; Kleppmann (2016). Architecture: next slide.

Work cells fork and the gate holds authority



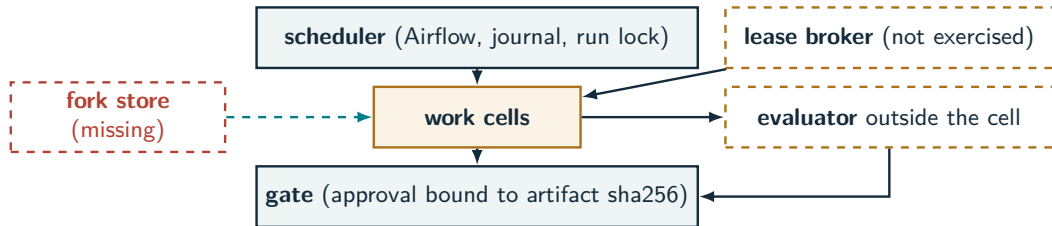
Our design documents: fork lifecycle and run recovery. Solid: built and ran. Dashed gold: not exercised (lease broker built; evaluator proposed). Dashed red: missing.

Work cells fork and the gate holds authority



Our design documents: fork lifecycle and run recovery. Solid: built and ran. Dashed gold: not exercised (lease broker built; evaluator proposed). Dashed red: missing.

Work cells fork and the gate holds authority



Fork contract, written before the fork (two of eight conditions shown)

5. never count siblings that reuse the same evidence as independent agreement

8. tear down every child, on any exit

observed in our second run

The fork contract was written before the fork store existed.

Our design documents: fork lifecycle and run recovery. Solid: built and ran. Dashed gold: not exercised (lease broker built; evaluator proposed). Dashed red: missing.

Promotion is small enough to model-check

A TLA+ design model of one cell, from epoch replacement to publication.

- evidence refers only to the frozen candidate
- approval requires evidence for that same candidate
- publication requires frozen candidate = evidence = approval under the current Airflow-bound epoch
- an external commit belongs to the current epoch
- a search-origin authority request cannot become an authority source
- an `in_doubt` effect has no blind retry and leaves that state only by observation

Design model: no TLC model-checker run and no trace check recorded yet. The epoch fence is proposed.

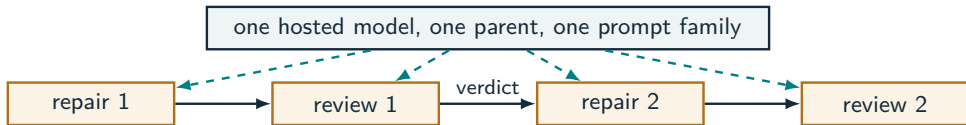
Dean & Barroso note that tail-tolerant techniques suit operations without critical mutations, while consistent updates use quorum algorithms such as Paxos.

Model-check the gate and count the children.

TLA+ authority model of our loop. Dean & Barroso, The tail at scale, CACM 2013 (“Mutations”).

The second review was not independent

Agreeing strangers are likely right, but nine sharing one map are one opinion.



Repair 2 answered review 1's verdict, and the same model graded it, a reused holdout (Dwork et al. 2015).

Choosing among 9 patches
needs a fresh test, as $\text{best-of-9} \leq 1 - P(\text{all fail})$.

Confirming one patch 9 times
is capped by what the runs share.

ANALOGY · BIOLOGY In our 2021 CRISPR-screen paper, each viability point had at least four technical replicates and each plot at least two independent experiments.

Nine forks of one parent are technical replicates.

Our experiments, Sep 2026: the records of review 1, repair 2 and review 2. Dwork et al., The reusable holdout, Science 2015. [Chen, arXiv:2606.27288 \(2026 preprint\)](#). Gilad, Eliaz et al., Commun. Biol. 4:399 (2021), Methods.

A reused grader becomes a training set

Even a protected evaluator can teach the search to overfit its feedback.



STATISTICS

A reused holdout overfits (Dwork et al. 2015), and the bias of adaptive analysis is bounded (Russo & Zou 2016).

RL TRAINING

Kimi K3 uses public verifiers for diagnostic feedback and hidden ones for held-out scenarios. A verbosity cap limits reward hacking by length.

PROPOSED Agents need a grader outside the child, every query logged, and a freeze before the final test.

Feedback that is optimised against stops being evidence.

Russo & Zou, AISTATS 2016. Dwork et al., Science 2015. Kimi K3 report §4.2.6 (verifiers), §4.1.2 (verbosity). Visible scorer: slide 15.

One formula, three fields

$$P(\text{at least one of } n) = 1 - (1 - p)^n$$

SYSTEMS

Dean & Barroso report each server is slow on 1 request in 100 (99th percentile 1 s), and a request fans out to 100 servers.

63%

of requests are slow.

BIOLOGY

Dorfman 1943: a pool tests clean only if every sample is. Poolkeh, two-step pools for ~ 9 M people, would need

295,951

tests, 30-fold fewer (a model, not a deployment).

THIS TALK

Suppose the race breaks 1 run in 10. Green runs needed before a miss is under 5%:

29

since $0.9^{29} = 0.047$.

All three assume independent draws.

Dean & Barroso, The tail at scale, CACM 2013. Dorfman, Ann. Math. Stat. 14:436 (1943). Eliaz, Danovich & Gasic, Poolkeh, medRxiv 2020 (Israel, $s_1 = 92$, $s_2 = 10$). Race: our first run, review 2. The 1-in-10 rate is illustrative.

Extra forks add few witnesses

100 forks give 9, 2 or 1 independent observations at $\rho = 0.1, 0.5, 1$.

$$\text{Var}(\bar{Y}_N) = \sigma^2 \left(\rho + \frac{1 - \rho}{N} \right)$$

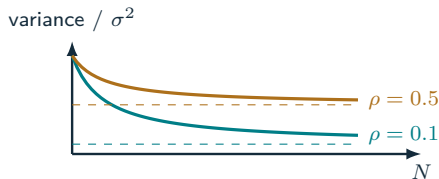
$$N_{\text{eff}} = \frac{N}{1 + (N - 1)\rho}$$

ρ : how much the copies share.

A little = 0.1, a lot = 0.5, identical = 1.

4 re-runs $\times$ 100 tests = 400 executions, 100 evidence IDs. Pool within a candidate and compare across.

Different models, both wrong: same answer 60% (chance 33%). 9 LLM judges: 2.2 independent votes.



Equal variances, common pairwise correlation $\rho \geq 0$, Kish (1965). 60%: Kim et al., ICML 2025 (HELM). 2.2: Kohli, arXiv:2605.29800 (2026 preprint), 9 judges, 7 families. Amdahl, AFIPS SJCC 1967.

Extra forks add few witnesses

100 forks give 9, 2 or 1 independent observations at $\rho = 0.1, 0.5, 1$.

$$\text{Var}(\bar{Y}_N) = \sigma^2 \left(\rho + \frac{1 - \rho}{N} \right)$$

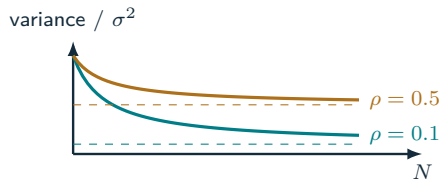
$$N_{\text{eff}} = \frac{N}{1 + (N - 1)\rho}$$

ρ : how much the copies share.

A little = 0.1, a lot = 0.5, identical = 1.

4 re-runs $\times$ 100 tests = 400 executions, 100 evidence IDs. Pool within a candidate and compare across.

Different models, both wrong: same answer 60% (chance 33%). 9 LLM judges: 2.2 independent votes.



Amdahl: speedup = $N / (1 + (N - 1)f)$, so 100 cores at 10% serial give $9.2\times$. ρ is the serial fraction of your evidence.

➤ N forks $\rightarrow N$ executions, but $N_{\text{eff}} < N$ independent observations.

Equal variances, common pairwise correlation $\rho \geq 0$, Kish (1965). 60%: Kim et al., ICML 2025 (HELM). 2.2: Kohli, arXiv:2605.29800 (2026 preprint), 9 judges, 7 families. Amdahl, AFIPS SJCC 1967.

The runtime holds some of the cluster labels

Methods for dependent evidence

design effect	Kish 1965
cluster-robust variance	Liang & Zeger 1986
dependent effect sizes	Hedges et al. 2010
lineage as annotation	Green et al. PODS'07

All of them need cluster labels that record which results share a parent, seed or test.

What the runtime can and cannot record

- ✓ records the shared parent, seed, test and fixture
- ✓ the egress gateway records the model each child called
- ✗ cannot see blind spots shared across different models (Kim et al.)

ANALOGY We inferred such sharing from outside in two studies. Address linking tied most bitcoin mined before February 2011 to 64 agents. Malignant cells cluster by patient tumour. A fork runtime records who shares what with every result.

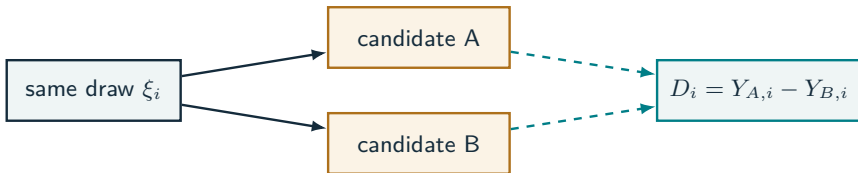
Other fields infer the clusters after the fact. A fork runtime can record them directly.

Blackburn et al., [arXiv:2206.02871 \(2022\)](#); Saurty-Seerunghen et al., [iScience 2026](#); Eliaz, [arXiv:2607.09689 §5](#).
Crossed factors: Cameron, Gelbach & Miller, [JBES 2011 \(multiway\)](#); [REStat 2008 \(wild-cluster bootstrap\)](#).

A fork can supply the control run

For a comparison, positive covariance helps.

$$\text{Var}(Y_A - Y_B) = \text{Var}(Y_A) + \text{Var}(Y_B) - 2 \text{Cov}(Y_A, Y_B)$$



PROPOSED The fork API should include a seed policy, `seed=copy|reseed` (slide 30).

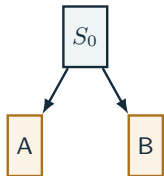
Test 2 relies on siblings and strangers sharing slot and host, so only ancestry differs (slide 45).

Share draws to compare and vary draws to corroborate.

Common random numbers: [Glasserman & Yao, Management Science \(1992\)](#).

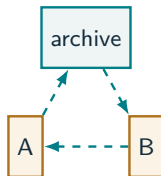
When a swarm gels

Execution ancestry



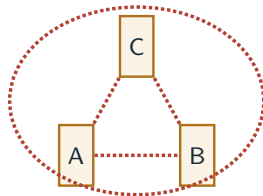
The state producing this child.

Evidence flow



Observations that influenced a decision.

Composition



Sets needing joint validation.

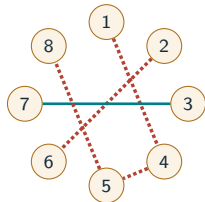
ANALOGY · PHYSICS Stockmayer (J. Chem. Phys. 11:45, 1943) found that enough cross-links join branched chains into one gel. Enough evidence edges can join separate lineages into one cluster.

Ancestry is a tree. Evidence flow is not.

Stockmayer (1943) is a precedent under idealised assumptions, and an agent threshold must come from the real dependency model.

Two passing patches can fail together

$$\Gamma_{ij} = U(\{i, j\}) - U(\{i\}) - U(\{j\}) + U(\emptyset)$$



Illustrative interaction graph

- fixed objective $U(A)$ for change set A on one parent.
- 8 candidates: 28 pair tests, 256 subsets.
- Effects among 3 or more patches can pass every pair test.
- Rebuild and evaluate every chosen composition (contract condition 7, end matter, verifies the merged result).

ANALOGY · BIOLOGY Genetics calls a strongly negative Γ synthetic lethality. We screened the genome for genes synthetically lethal with an SRC-3 inhibitor (Commun. Biol. 4:399, 2021).

Rebuild and evaluate the composition you ship.

Workers return a receipt of evidence and origin

$$r_k = (\hat{\theta}_k, J_k, n_k, \mathcal{E}_k, \mathcal{L}_k, m_k)$$

result $(\hat{\theta}, J, n)$, evidence $(\mathcal{E})$, origin $(\mathcal{L}, m)$

BUILT

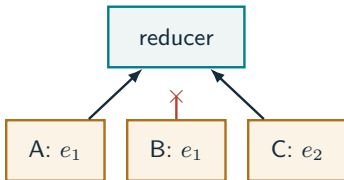
- numeric summaries merge in any tree order
- a repeated evidence ID stops the merge
- lineage travels with the result and is not yet used

PROPOSED

- each evidence ID is one execution, so a retry or hedge is never counted twice
- if sharing is unknown, the runtime abstains

“Worker count by itself carries no statistical information.”

arXiv:2607.09689, reduction invariant



duplicate e_1 stops the merge

MEASURED 4-worker trace took 6.70 s end to end, pooled 4.942 vs. full-sample 4.945.

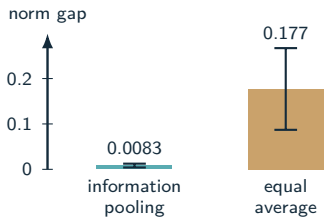
The estimator is Cochran's, but its contract was missing.

Eliaz, [arXiv:2607.09689](https://arxiv.org/abs/2607.09689) (2026), §2–5, trace §4. Cochran (1954). DeepSeek-V4 §5.2.5 (DSec).

A sandbox cannot verify a claim

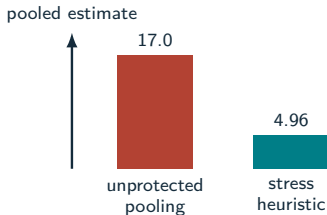
Honest, unequal shards

Five logistic shards, $n = 60$ to $5,000$. Distance to the full-data MLE, lower is better.



One shard forges its precision

A 2,000-point shard reports a distant estimate, precision inflated a further $50\times$.



Mean $\pm$ SD over 8 fixed seeds, not CIs. Sample-size weighting is not yet compared. The heuristic is no Byzantine guarantee.

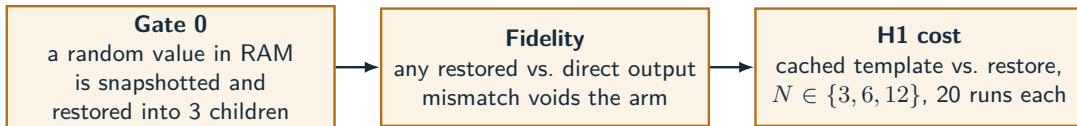
PROPOSED The controller sets precision ($n \cdot J$) by re-running or re-scoring on held-out data.

The controller should set precision for each child.

Synthetic checks, Eliaz, [arXiv:2607.09689](https://arxiv.org/abs/2607.09689), §4, not agent-task benchmarks.

Test 1: restore versus a warm cache

Our claim fails if the whole interval for the gain is under 10 s at any N .

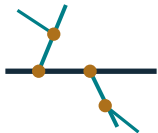


A result in which the build cache wins is a valid outcome.

Pre-registration, tags prereg-v1 to prereg-v3, github.com/zozo123/cam-talk-london-26. Restore cost follows REAP (ASPLOS'21) and FaaSnap (EuroSys'22). Decision rules in full are in the end matter.

Why siblings may fail together

In simulated actomyosin, the Arp2/3 level changes collective behaviour.



schematic, gold = Arp2/3

Arp2/3 level	Reported network response
High	stalls
Low	contracts
Intermediate	loosely connected clusters may collapse suddenly in motor-driven avalanches, similar to experimental “cytoquakes”

We forecast these avalanches with network science and machine learning (Li et al., JPCB 2021).

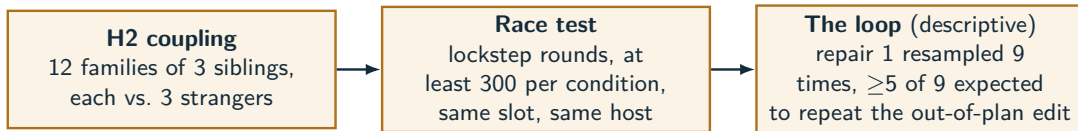
This is motivation only.

Analogy: this does not predict software behaviour.

Team results: [Liman et al. \(PNAS, 2020\)](#), [Li et al. \(JPCB, 2021\)](#), [Eliaz et al. \(PRE, 2020, linker valency\)](#).

Test 2: sibling failure coupling

Our claim fails if the sibling excess $\Delta\rho$ is confidently below 0.05.



$\Delta\rho$ is how much more siblings fail together than strangers.

Siblings are technical replicates. Strangers are independent experiments, matched on slot and host.

A correlation excess of 0.05 reduces 9 siblings to 6.4 independent witnesses.

Each rejection rule was written before the data.

Pre-registration, tags prereg-v1 to prereg-v3. Replicates follow Gilad, Eliaz et al., Commun. Biol. 2021. Decision rules in full are in the end matter.

What two tests cannot separate

A positive result does not identify its mechanism.

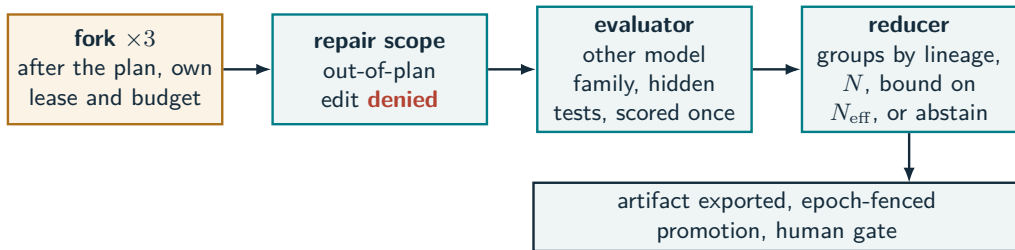
Vary	Observe	Question
Prefix cost and dirty pages	Resource use and memory	Condition under which reuse pays
No sharing, archive, or routing	Quality at equal full cost	Condition under which communication helps
Evidence overlap and coupling	Calibration and error	Whether the reducer overcounts
Evaluation budget	False acceptance and time to target	Best allocation of compute
Composition order and size	Joint success and regressions	Which interactions are significant

A fast fork with a worse search policy can still lose to a slower fork with a better one.

Separate a runtime gain from a search-policy or evaluation advantage.

Ablation plan (proposed). The number of tries as part of the result follows Stroebel, Kapoor & Narayanan, arXiv:2411.17501.

The same work order, run as designed



Review 2 would not count as a second witness.

Search may run in parallel. Promotion may not.

A design that has not been run. Three tries, because when a wrong answer costs more than none the best number is often under ten (Stroebl et al. 2024). Hidden verifiers isolated from agents appear in the Kimi K3 report §4.2.6.

Six open problems for the prototype

1. **Dependence from lineage and evidence edges.** Estimate ρ , the amount of evidence that forks share, from fork trees and evidence manifests.
2. **Leases that fork.** Capabilities should be re-issued on restore, be revocable, and never be ambient.
3. **Warm state and side channels.** Place forks on hosts by trust level as well as by utilisation.
4. **Evidence commit.** Effects on remote services and on people should be held until commit.
5. **Layers or VM fork.** Determine when a content-addressed build is faster than a live fork.
6. **Human-gate latency.** The sandbox should be snapshotted, destroyed and restored across a wait of one day.

We ask for help with the first two problems.

Canary requests from the hedging paper

“a particular request exercises an untested code path”

Dean & Barroso, The Tail at Scale, CACM 2013,
“Canary requests”

- ✗ Thousands of servers crash or delay at once.
- ✓ Sent to one or two leaf servers first, the rest only if the canary succeeds

A proposed correlation canary runs a few siblings and strangers before a hundred forks.
Agreement is no proof, as 9 frontier judges who all agreed were wrong 9.1% of the time (Kohli).

Hedges mask slow replicas, and canaries contain shared failures.

Dean & Barroso, CACM 56(2):74–80, 2013, p. 79. Our experiments, Sep 2026, reviews 1 and 2. [Kohli, arXiv:2605.29800 \(2026 preprint\)](#).

Our first run
new code, no test of its own



nine green forks would
share the gap (hypothetical)

Hedging works when failures are independent.

Contributions

- BUILT** a receipt contract in which reused evidence stops the merge and lineage travels with every result
- MEASURED** two experiments with the loop
- PROPOSED** Hypothesis H1: restore beats a warm cache by more than 10s
- PROPOSED** Hypothesis H2: siblings from one snapshot fail together more often than strangers

2009: else: VM_wait(ID)

2026: else: reduce(receipts) $\rightarrow N_{\text{eff}}$, or abstain (proposed)

 N forks $\rightarrow N$ executions, but $N_{\text{eff}} < N$

independent observations.

We request help with estimating ρ from lineage and with leases that fork.

arXiv:2607.09689

Fork lineage, in full

	System	What it forks or moves	Reported
2003	Xen (Cambridge)	paravirtualised VMs	base of the Xen rows below
2005	Live migration (Cambridge)	pre-copied memory, to another host	60 ms downtime, Quake 3 server
2005	Potemkin	honeypot VMs, CoW memory	521 ms clone to first reply
2009	SnowFlock	VM_fork(N) across hosts, on Xen	600–800 ms
2020	Catalyzer	sfork of a running gVisor sandbox	under 1 ms, best case
2023	Nephele; MITOSIS	unikernel VMs on Xen; fork over RDMA	—
2026	DeltaBox Shepherd Crab Planarian Kimi K3 AgentENV (training)	CoW files + incremental CRIU overlay + backend checkpoints eBPF effect classes per turn local and remote statepoints Firecracker pause, fork, snapshot	10.83 ms checkpoint fork 134–143 ms, flat from 42 MB to 5.8 GB over 75% of turns need no checkpoint remote rollback by compensating actions checkpoint as low as 133 ms, resume 49 ms

SOSP'03; NSDI'05; SOSP'05; EuroSys'09; ASPLOS'20; EuroSys'23; OSDI'23; [DeltaBox](#); [Shepherd](#); [Crab](#); [Planarian](#); [Kimi K3](#) §5.3.2.

Fork cost model and API timings

Cost model PROPOSED

$$C_{\text{cold}} = N(P + W + V)$$

$$C_{\text{fork}} = P + H + N(R + D + W + V)$$

$$(N - 1)P > H + N(R + D)$$

prepare P , capture H , restore R , diverge D , work W , evaluate V . W includes model calls and cancels. Each child still rebuilds its patch.

Three API paths MEASURED

	Operation	Success	p50	p95
Daytona	create	1024/1024	0.20 s	1.12 s
Tensorlake	create	256/256	3.44 s	9.00 s
Sandbox platform	restore + run + capture	255/256	6.87 s	9.04 s

Sandbox platform: 141 MB named snapshot, concurrency 12. Different operations: not a ranking. Teardown excluded. Memory capture unverified, so that row is restore fan-out.

Eliaz, [arXiv:2607.09689](https://arxiv.org/abs/2607.09689), Table 2.

The fork contract, in full

Fork contract, written before the fork

1. fork from an immutable, content-addressed parent
2. each branch: own lease, TTL, budget, provenance
3. no publishing credentials in a coding cell
4. record parent checkpoint, branch ID, evidence IDs
5. **never count siblings that reuse the same evidence as independent agreement**
6. merge deterministically; conflicts are evidence
7. full verification and review on the merged result
8. tear down every child, on any exit our second run

Airflow Factory fork contract (design doc). Slide 31 shows conditions 5 and 8.

Pre-registered decision rules

Test 1: does restore beat a warm cache?

Step	Design	Decision
Gate 0	a RAM nonce, snapshot, restore 3 children: does it survive?	if not, every result is “restore fan-out”
Fidelity	deterministic tests, restored vs. direct	any output mismatch voids the arm
H1 cost	cached template vs. restore, cold as reference; $N \in \{3, 6, 12\}$; 20 interleaved reps per arm and N ; makespan to first test results, one clock for both arms; no model calls	supports if the 95% CI of the median gain is above 10 s at every N ; rejects if the 98.3% (Bonferroni) CI is below 10 s at any N

Test 2: do siblings fail together?

Step	Design	Decision
H2 coupling	12 snapshot families $\times$ 3 siblings, each paired with 3 strangers restored from other families, same slot, same host; the race test in lockstep rounds, at least 300 per condition	supports if a one-sided sign-flip test gives $p < 0.05$ and $\Delta\rho > 0.05$; rejects if the 90% CI of $\Delta\rho$ is below 0.05; else inconclusive
The loop	repair 1 re-sampled 9 times from its recorded input; the only model calls	descriptive; predicted: ≥ 5 of 9 repeat the out-of-plan edit

Pre-registration in github.com/zozo123/cam-talk-london-26, tags prereg-v1 to prereg-v3.

Research record: biophysics

PhD, University of Houston, with the Rice Center for Theoretical Biological Physics

Eliaz, Nedelec, Morrison, Levine & Cheung. Insights from graph theory on the morphologies of actomyosin networks with multilinkers. *Phys. Rev. E* 102:062420 (2020). First author.

Liman, . . . , **Eliaz**, . . . , Cheung. The role of the Arp2/3 complex in shaping the dynamics and structures of branched actomyosin networks. *PNAS* 117:10825 (2020).

Li, Liman, **Eliaz** & Cheung. Forecasting avalanches in branched actomyosin networks with network science and machine learning. *J. Phys. Chem. B* 125:11591 (2021).

Zegarra, . . . , **Eliaz**, Impact of hydrodynamic interactions on protein folding rates depends on temperature. *Phys. Rev. E* 97:032402 (2018).

Zhang, Nde, **Eliaz**, Jennings, Cieplak & Cheung. CaXML: Chemistry-informed machine learning explains mutual changes between protein conformations and calcium ions in calcium-binding proteins using structural and topological features. *Protein Science* 34:e70023 (2025).

Thesis

Eliaz. Physics of self-assembly in complex matter. PhD thesis, University of Houston (2020); self-published in book form as *Physics of Complex (Biological) Matter* (2023).

Full author lists and positions: CLAIM-FENCE.md. Meeting abstracts (Biophysical Society, APS March Meeting 2018–2021) are not listed as papers.

Research record: genomics

Genome architecture and bioinformatics

Harris, . . . , **Eliaz**, . . . , Lieberman Aiden, Rowley. Chromatin alternates between A and B compartments at kilobase scale for subgenic organization. *Nat. Commun.* 14:3303 (2023).

Kaushal, . . . , **Eliaz**, CTCF loss has limited effects on global genome architecture in *Drosophila* despite critical regulatory functions. *Nat. Commun.* 12:1011 (2021).

Barkai, . . . , **Eliaz**, OffRisk: a docker image for annotating CRISPR off-target sites in the human genome. *Bioinformatics Advances* 3:vbad138 (2023).

Cancer genomics

Saurty-Seerunghen, El-Habr, **Eliaz**, . . . , Junier. A unique malignant cell type per patient tumor encoded in each cancer cell transcriptome. *iScience* 29:115139 (2026).

Gilad, **Eliaz**, A genome-scale CRISPR Cas9 dropout screen identifies synthetically lethal targets in SRC-3 inhibited cancer cells. *Commun. Biol.* 4:399 (2021).

Gilad, **Eliaz**, Reply to: Target expression is a relevant factor in synthetic lethal screens. *Commun. Biol.* 5:836 (2022).

Gilad, **Eliaz**, Drug-induced PD-L1 expression and cell stress response in breast cancer cells can be balanced by drug combination. *Sci. Rep.* 9:15099 (2019).

Full author lists and positions: CLAIM-FENCE.md.

Research record: preprints, systems and talks

Preprints

Eliaz. Evidence-Aware MapReduce for Forkable Compute. arXiv:2607.09689 (2026). Sole author; code github.com/zozo123/boltzmann-mapreduce. Blackburn, Huber, **Eliaz**, . . . , Lieberman Aiden. Cooperation among an anonymous group protected Bitcoin during failures of decentralization. arXiv:2206.02871 (2022). **Eliaz**, Danovich & Gasic. Poolkeh finds the optimal pooling strategy for a population-wide COVID-19 testing. medRxiv (2020). First author. Hitz, . . . , **Eliaz**, . . . , Cherry. The ENCODE uniform analysis pipelines. bioRxiv (2023). The ENCODE Project Consortium (member). The Encyclopedia of DNA Elements. bioRxiv (2026). Chamo & **Eliaz**. Continuation geometry resolves apparent branch splitting in unequal-mass three-body orbits. ai.viXra:2608.0069 (2026); not peer reviewed.

Systems and open source

Airflow Factory, an open-source Apache Airflow software factory: github.com/zozo123/ariflow-swfactory. This talk's slides and notes: github.com/zozo123/cam-talk-london-26. apache/airflow: sandbox toolset with Docker sbx and OpenSandbox backends (merged, 2026); two more backends (open). openclaw/crabbox, nushell, and sandbox PRs to Spark, Kafka Connect and mecatl (2026). The Sandbox Shift: a field manual for running untrusted code (2026).

Talks, October 2026

Cambridge SRG (this talk), 15 Oct.
RustChinaConf, Shenzhen: Disposable Runners, Warm Cargo Factory, 17 Oct.
Zenity AI Agent Security Summit, New York: Your Agent Escaped Without Escaping the Sandbox, 21 Oct.

References: systems

Fork, snapshot, isolation

Barham et al. Xen and the art of virtualization. SOSP 2003.
Vrable et al. Scalability, fidelity, and containment in the Potemkin virtual honeyfarm. SOSP 2005.
Lagar-Cavilla et al. SnowFlock. EuroSys 2009.
Madhavapeddy et al. Jitsu. NSDI 2015.
Manco et al. My VM is lighter (and safer) than your container. SOSP 2017.
Baumann et al. A fork() in the road. HotOS 2019.
Agache et al. Firecracker. NSDI 2020.
Du et al. Catalyzer. ASPLOS 2020.
Brooker et al. Restoring uniqueness in microVM snapshots. arXiv:2102.12892, 2021.
Ustiugov et al. Benchmarking, analysis, and optimization of serverless function snapshots (REAP). ASPLOS 2021.
Lupu et al. Nephele: cloning unikernel-based VMs. EuroSys 2023.
Clark et al. Live migration of virtual machines. NSDI 2005.

Leases and fencing

Gray & Cheriton. Leases. SOSP 1989.
Burrows. The Chubby lock service. OSDI 2006.
Kleppmann. How to do distributed locking. 2016.

Agent sandboxes, 2026

Dong et al. DeltaBox. arXiv:2605.22781. Yu et al. Shepherd. arXiv:2605.10913. Wu et al. Crab. arXiv:2604.28138.
Guo et al. Planarian: managing agent state with statepoints. arXiv:2609.35366. Zheng et al. arXiv:2608.22928.
Kimi Team (Moonshot AI). Kimi K3: Open Frontier Intelligence. Tech. report, 2026, §4.1.2, §4.2.4, §4.2.6, §5.3.2. DeepSeek-AI. DeepSeek-V4 (DSeC). arXiv:2606.19348.
Wei et al. No provisioned concurrency: fast RDMA-codesigned remote fork (MITOSIS). OSDI 2023. Ao et al. FaaSnap. EuroSys 2022.

Effects, authority, provenance

Nightingale, Chen & Flinn. Speculative execution in a distributed file system. SOSP 2005.
Nightingale et al. Rethink the sync. OSDI 2006.
Cully et al. Remus. NSDI 2008.
Watson et al. Capsicum. USENIX Security 2010.
Watson et al. CHERI. IEEE S&P 2015.
Birgisson et al. Macaroons. NDSS 2014.
Muniswamy-Reddy et al. Provenance-aware storage systems. USENIX ATC 2006.
Pasquier et al. Practical whole-system provenance capture. SoCC 2017.
Torres-Arias et al. in-toto. USENIX Security 2019.
O'Callahan et al. Engineering record and replay for deployability. USENIX ATC 2017.

References: evidence and evaluation

Dependent evidence

Dorfman. The detection of defective members of large populations. *Ann. Math. Stat.* 14(4):436–440, 1943.

Cochran. The combination of estimates from different experiments. *Biometrics* 1954.

Kish. *Survey Sampling*. Wiley, 1965.

Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS SJCC* 1967.

Liang & Zeger. Longitudinal data analysis using generalized linear models. *Biometrika* 1986.

Hedges, Tipton & Johnson. Robust variance estimation in meta-regression with dependent effect size estimates. *Research Synthesis Methods* 2010.

Green, Karvounarakis & Tannen. Provenance semirings. *PODS* 2007.

Cameron, Gelbach & Miller. Bootstrap-based improvements for inference with clustered errors. *REStat* 2008.

Cameron, Gelbach & Miller. Robust inference with multiway clustering. *JBES* 2011.

Glasserman & Yao. Some guidelines and guarantees for common random numbers. *Management Science* 1992.

Stockmayer. Theory of molecular size distribution and gel formation in branched-chain polymers. *J. Chem. Phys.* 11:45, 1943.

Shared errors across models

Kim, Garg, Peng & Garg. Correlated errors in large language models. *ICML 2025*, PMLR 267. [arXiv:2506.07962](#).

Goel et al. Great models think alike and this undermines AI oversight. *ICML 2025*. [arXiv:2502.04313](#).

Evaluation under search

Dean & Barroso. The tail at scale. *CACM* 2013 (hedged requests; also fan-out, good enough, canary requests, mutations).

Dean. Designs, lessons and advice from building large distributed systems (“Numbers everyone should know”). *LADIS keynote*, 2009.

Brown et al. Large Language Monkeys: scaling inference compute with repeated sampling. [arXiv:2407.21787](#), 2024.

Von Arx, Chan & Barnes (METR). Recent frontier models are reward hacking. 5 June 2025.

Zhong, Raghunathan & Carlini. ImpossibleBench. [arXiv:2510.20270](#), 2025.

Dwork et al. The reusable holdout. *Science* 2015.

Stroebl, Kapoor & Narayanan. The limits of inference scaling through resampling (v1: Inference scaling fLaws). [arXiv:2411.17501](#), 2024.

Russo & Zou. Controlling bias in adaptive data analysis using information theory. *AISTATS* 2016.

Shared errors across models: 2026 preprints

Jo, Garg & Raghavan. The subjectivity of monoculture. [arXiv:2602.24086](#), preprint.

Kohli. Nine judges, two effective votes: correlated errors undermine LLM evaluation panels. [arXiv:2605.29800](#), preprint.

Chen. When does combining language models help? A co-failure ceiling on routing, voting, and mixture-of-agents across 67 frontier models. [arXiv:2606.27288](#), preprint.

Bone, Stephany & del Rio-Chanona. Monocultural biases: correlated biases in large language models lead to unequal systemic exclusion rates in hiring. [arXiv:2609.22169](#), preprint.

References: own work and documentation

Speaker and co-authors

Y. Eliaz. Evidence-Aware MapReduce for Forkable Compute. arXiv:2607.09689, 2026.

github.com/zozo123/ariflow-swfactory, runs SELFHOST-2/3, 27 Sep 2026.

Blackburn, Huber, Eliaz et al. Cooperation among an anonymous group protected Bitcoin during failures of decentralization. arXiv:2206.02871, 2022.

Saurty-Seerunghen, El-Habr, Eliaz et al. A unique malignant cell type per patient tumor encoded in each cancer cell transcriptome. iScience 29:115139, 2026.

Hitz, . . . , Eliaz, . . . , Cherry. The ENCODE uniform analysis pipelines. bioRxiv 2023, doi:10.1101/2023.04.04.535623.

Gilad, Eliaz et al. A genome-scale CRISPR Cas9 dropout screen identifies synthetically lethal targets in SRC-3 inhibited cancer cells. Commun. Biol. 4:399, 2021.

Barkai, . . . , Eliaz, OffRisk. Bioinformatics Advances 3:vbad138, 2023.

Eliaz, Danovich & Gasic. Poolkeh. medRxiv 2020.

Speaker's physics work (slides 7 and 44)

Eliaz et al. PRE 102:062420, 2020; Liman et al. PNAS 117:10825, 2020; Li et al. JPCB 125:11591, 2021.

Documentation

OBuilder (github.com/ocurrent/obuilder); day10 (tunbury.org, 16 Feb 2026); Docker Sandboxes security; Firecracker snapshotted, snapshot-support and prod-host-setup docs; CRIU TCP_REPAIR; gVisor checkpoint/restore.

RL and exploration

Ecoffet et al. Go-Explore. Nature 2021.